

Join the lecture online on your dashboard.

Neural Network Optimization

Modifying Gradient Descent for Non-Convex Surfaces

```
class sklearn.neural_network.MLPClassifier(hidden_layer_sizes=(100,),
activation='relu', *, solver='adam', alpha=0.0001, batch_size='auto',
learning_rate='constant', learning_rate_init=0.001, power_t=0.5, max_iter=200,
shuffle=True, random_state=None, tol=0.0001, verbose=False, warm_start=False,
momentum=0.9, nesterovs_momentum=True, early_stopping=False,
validation_fraction=0.1, beta_1=0.9, beta_2=0.999, epsilon=1e-08,
n_iter_no_change=10, max_fun=15000)
```

[\[source\]](#)

momentum : float, default=0.9

Momentum for gradient descent update. Should be between 0 and 1. Only used when solver='sgd'.

nesterovs_momentum : bool, default=True

Whether to use Nesterov's momentum. Only used when solver='sgd' and momentum > 0.

solver : {'lbfgs', 'sgd', 'adam'}, default='adam'

The solver for weight optimization.

- 'lbfgs' is an optimizer in the family of quasi-Newton methods.
- 'sgd' refers to stochastic gradient descent.
- 'adam' refers to a stochastic gradient-based optimizer proposed by Kingma, Diederik, and Jimmy Ba

batch_size : int, default='auto'

Size of minibatches for stochastic optimizers. If the solver is 'lbfgs', the classifier will not use minibatch. When set to "auto", `batch_size=min(200, n_samples)`.

learning_rate : {'constant', 'invscaling', 'adaptive'}, default='constant'

Learning rate schedule for weight updates.

- 'constant' is a constant learning rate given by 'learning_rate_init'.
- 'invscaling' gradually decreases the learning rate at each time step 't' using an inverse scaling exponent of 'power_t'. `effective_learning_rate = learning_rate_init / pow(t, power_t)`
- 'adaptive' keeps the learning rate constant to 'learning_rate_init' as long as training loss keeps decreasing. Each time two consecutive epochs fail to decrease training loss by at least tol, or fail to increase validation score by at least tol if 'early_stopping' is on, the current learning rate is divided by 5.

Only used when `solver='sgd'`.

When life is simple

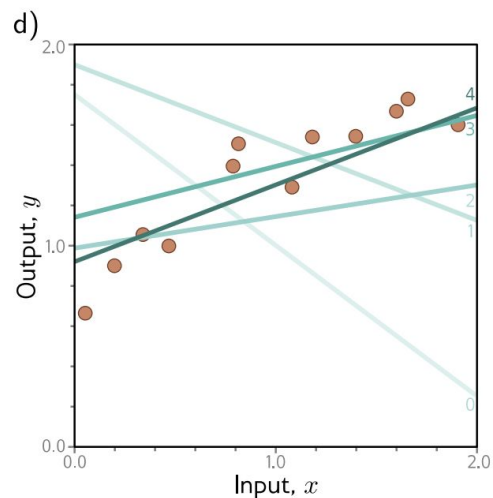
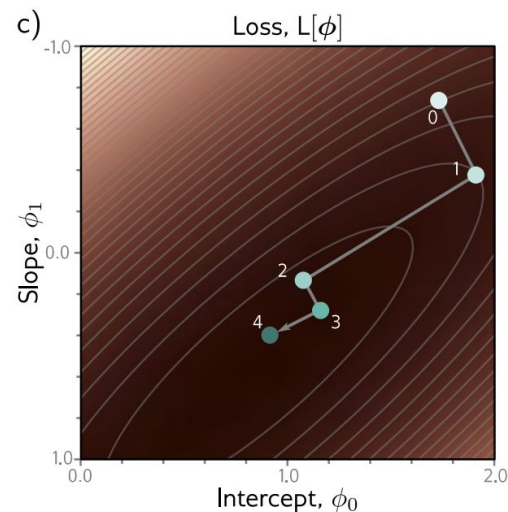
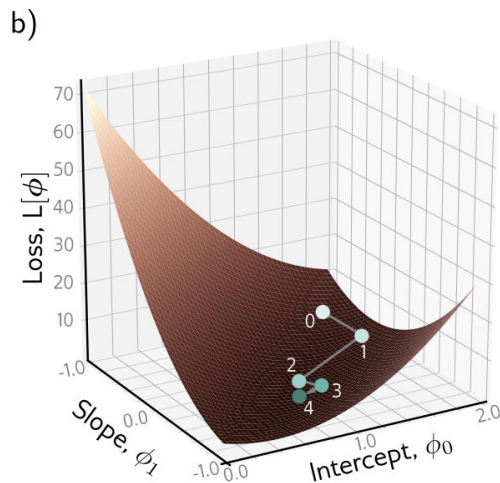
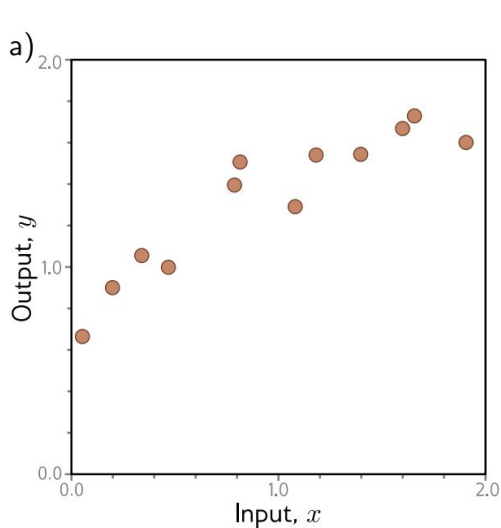


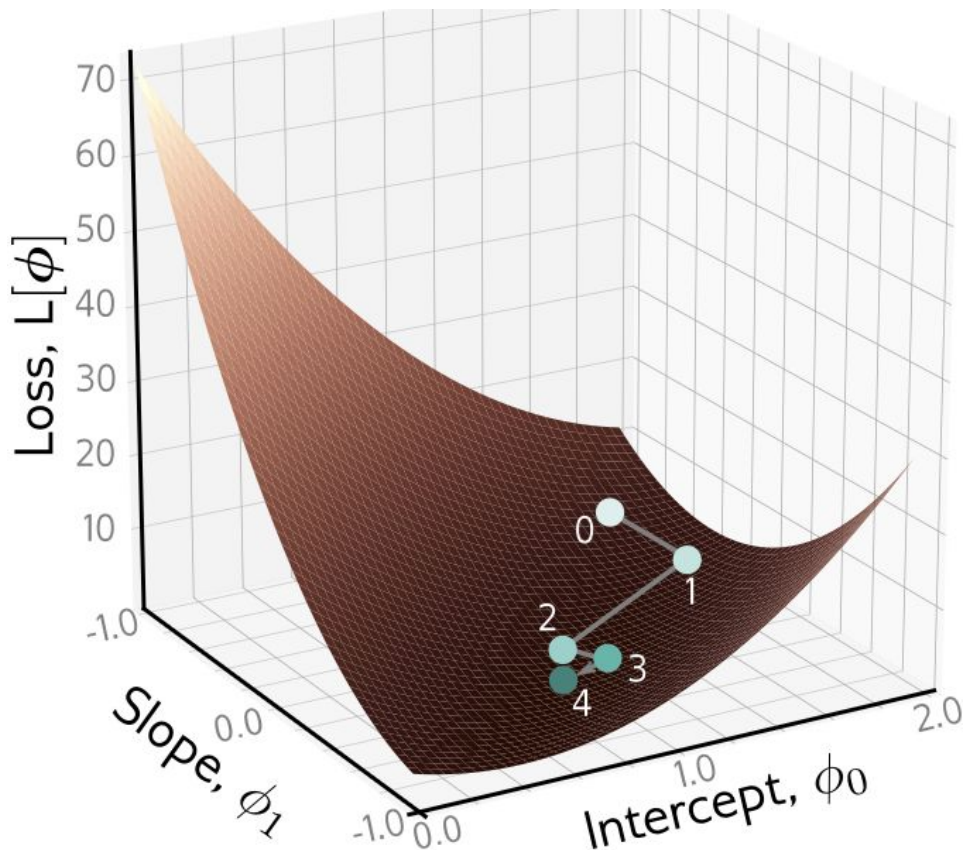
Figure 6.1 Gradient descent for the linear regression model. a) Training set of $I = 12$ input/output pairs $\{x_i, y_i\}$. b) Loss function showing iterations of gradient descent. We start at point 0 and move in the steepest downhill direction until we can improve no further to arrive at point 1. We then repeat this procedure. We measure the gradient at point 1 and move downhill to point 2 and so on. c) This can be visualized better as a heatmap, where the brightness represents the loss. After only four iterations, we are already close to the minimum. d) The model with the parameters at point 0 (lightest line) describes the data very badly, but each successive iteration improves the fit. The model with the parameters at point 4 (darkest line) is already a reasonable description of the training data.

The convex loss landscape

Convexity: every chord line segment between two points on the surface) lies above the function and does not intersect it.

Convexity implies that wherever we initialize the parameters, we are bound to reach the minimum if we keep walking downhill; the training procedure can't fail.

What does the loss landscape for nonlinear problems look like ?

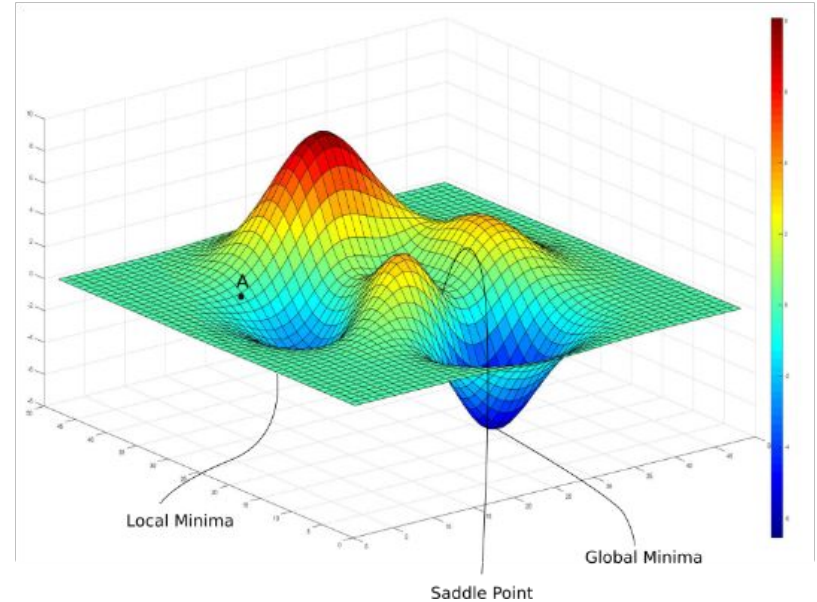


Non-Linear Loss Landscape

What is different here ?

1. Presence of local minima
2. Presence of saddle points

Let's have a look at how we find the optimal parameter values.



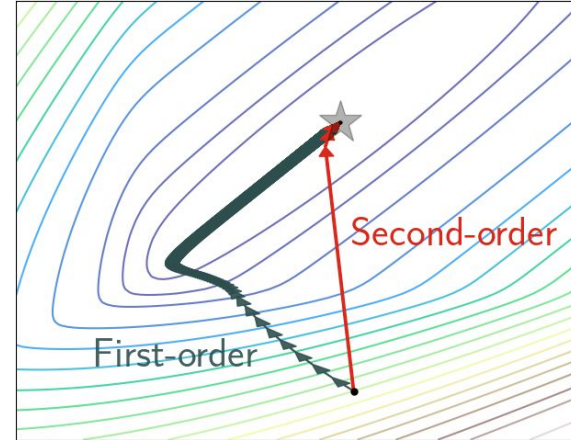
Philosophy behind NN Optimization

The **goal** of an optimization algorithm is to **find parameters** that **minimize the loss**.

Standard methods for training neural networks are **iterative**. These algorithms **initialize the parameters heuristically** and then **adjust them** repeatedly in such a way that the **loss decreases**.

$$\hat{\phi} = \operatorname{argmin}_{\phi} [L[\phi]].$$

https://maximerobeyns.com/second_order_opt



[GitHub - lilipads/gradient_descent_viz: interactive visualization of 5 popular gradient descent methods with step-by-step illustration and hyperparameter tuning UI](#)

First Order Methods

Gradient Descent: The Foundation

Narrative: To reach the bottom of the valley, we follow the steepest slope downhill.

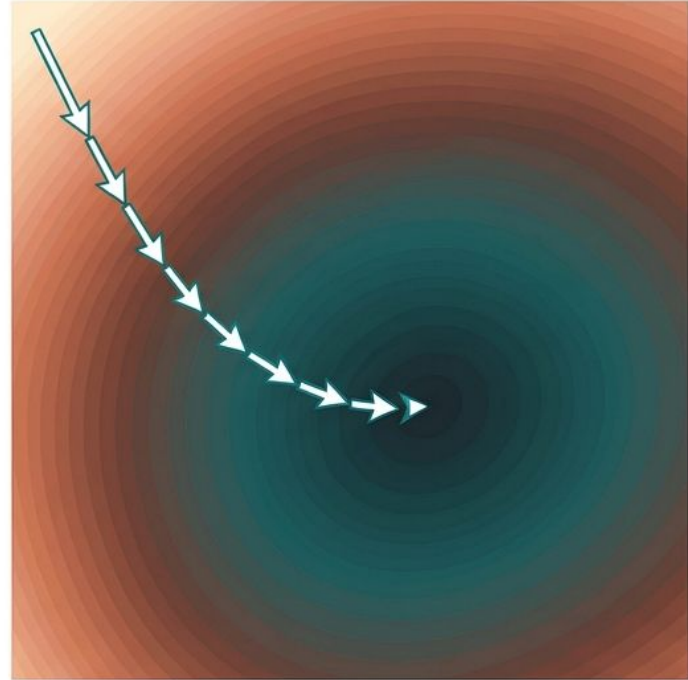
Algorithm:

1. Calculate Gradient: $\frac{\partial L}{\partial \phi}$ (This vector points uphill).
2. Update Step: Move in the opposite direction.

The Update Rule:

$$\phi \leftarrow \phi - \alpha \cdot \frac{\partial L}{\partial \phi}$$

Note: α is the Learning Rate. It determines the step size.



Gradient Descent

The simplest method we will discuss is gradient descent. This starts with initial parameters $\phi = [\phi_0, \phi_1, \dots, \phi_N]^T$ and iterates two steps:

Step 1. Compute the derivatives of the loss with respect to the parameters:

$$\frac{\partial L}{\partial \phi} = \begin{bmatrix} \frac{\partial L}{\partial \phi_0} \\ \frac{\partial L}{\partial \phi_1} \\ \vdots \\ \frac{\partial L}{\partial \phi_N} \end{bmatrix}. \quad (6.2)$$

Step 2. Update the parameters according to the rule:

$$\phi \longleftarrow \phi - \alpha \cdot \frac{\partial L}{\partial \phi}, \quad (6.3)$$

where the positive scalar α determines the magnitude of the change.

Gradient Descent on Linear Model

The linear model $f[x, \phi]$ maps a scalar input x to a scalar output y and has parameters $\phi = [\phi_0, \phi_1]^\top$, which represent the y-intercept and the slope:

$$\begin{aligned} y &= f[x, \phi] \\ &= \phi_0 + \phi_1 x. \end{aligned}$$

Given a dataset $\{x_i, y_i\}$ containing I input/output pairs, we choose the least squares loss function:

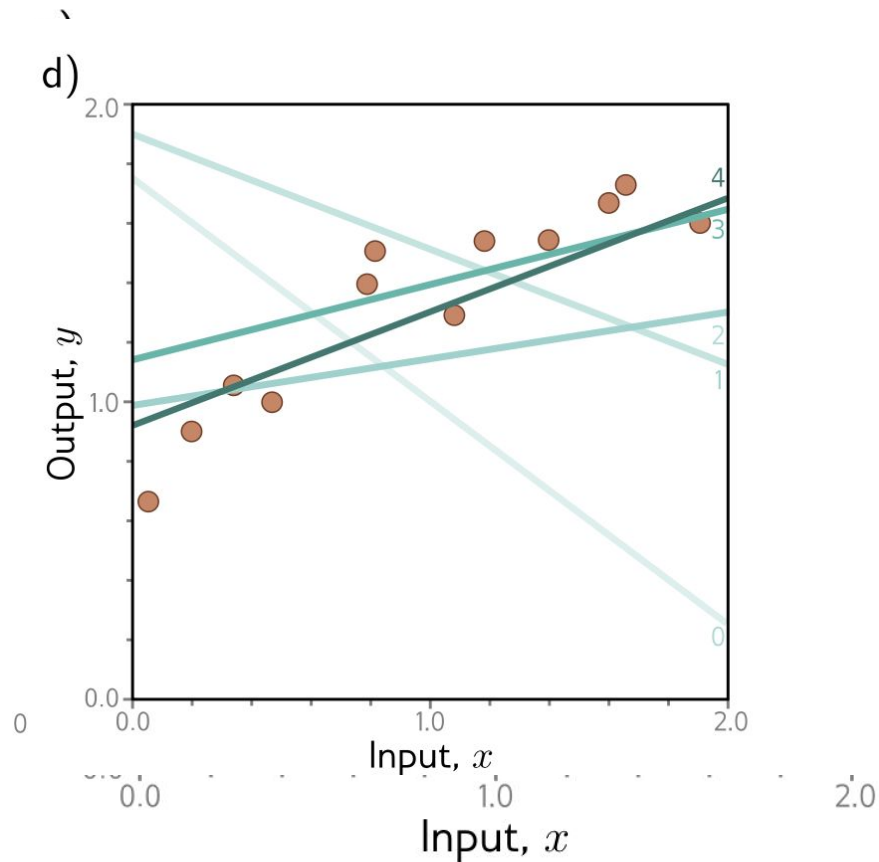
$$\begin{aligned} L[\phi] &= \sum_{i=1}^I \ell_i = \sum_{i=1}^I (f[x_i, \phi] - y_i)^2 \\ &= \sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2 \end{aligned}$$

Gradient Descent on Linear Model

The dataset

The loss landscape and path of gradient descent

Performance of model at each step



Gradient Descent on Linear Model

Summary:

1. Linear regression problems have a convex loss landscape
2. No matter where we initialize our model, even a basic step gradient descent optimization function will take us to global minima
3. What do we think happens for more complex loss landscapes with valleys and ridges?

Let's add some complexity to our loss landscape and see how step gradient descent performs.

The Reality of Non-Convexity

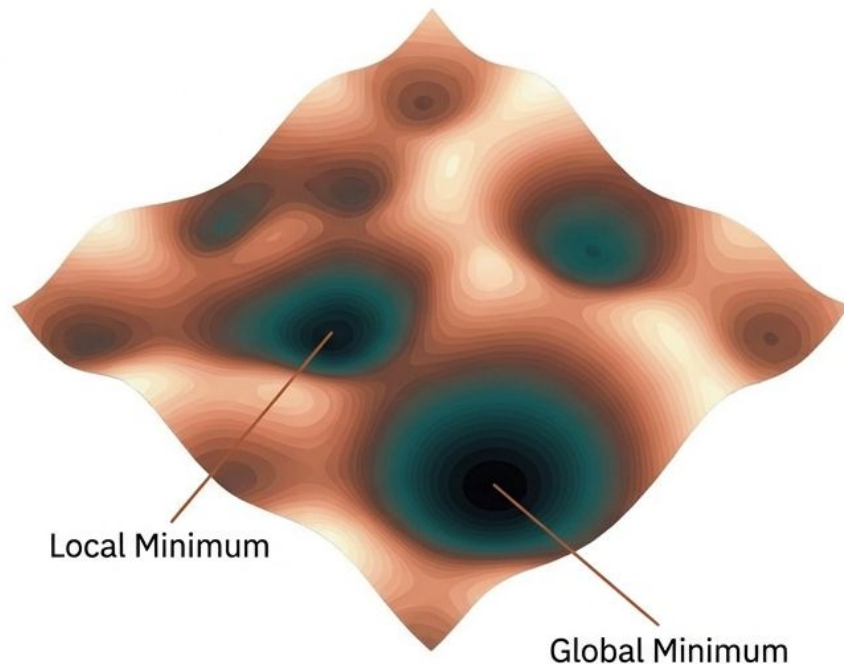
Convex vs. Non-Convex

Linear Regression is Convex: Like a simple bowl. It has a guaranteed global minimum.

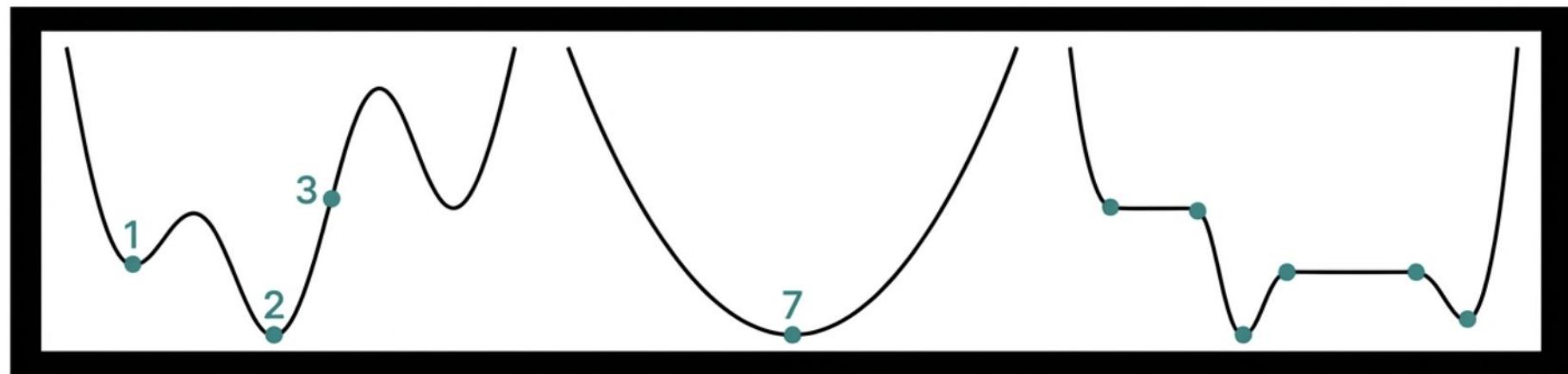
Deep Networks are Non-Convex: Complex terrain with multiple traps.

The Risks

- **Local Minima:** Valleys that are not the lowest point. Gradient is zero; standard GD stops here.
- **Saddle Points:** Areas flat in one direction but curved in another.



Problem 6.6: Reading the Loss Landscape



Function A (Left)	Function B (Center)	Function C (Right)
Non-Convex. Contains local minima.	Convex. Bowl shape. Single Global Minimum.	Non-Convex. Contains plateaus.

Point Analysis:

- Point 1: Local Minimum
- Point 2: Global Minimum
- Point 3: Neither (Gradient $\neq 0$)
- Point 7: Global Minimum (Convex)

Introduction to Gabor Model

The Gabor model maps scalar input x to scalar output y and consists of a sinusoidal component (creating an oscillatory function) multiplied by a negative exponential component (causing the amplitude to decrease as we move from the center).

$$f[x, \phi] = \sin[\phi_0 + 0.06 \cdot \phi_1 x] \cdot \exp\left(-\frac{(\phi_0 + 0.06 \cdot \phi_1 x)^2}{32.0}\right)$$

How many parameters does this model have ?

What are their geometric implications ?

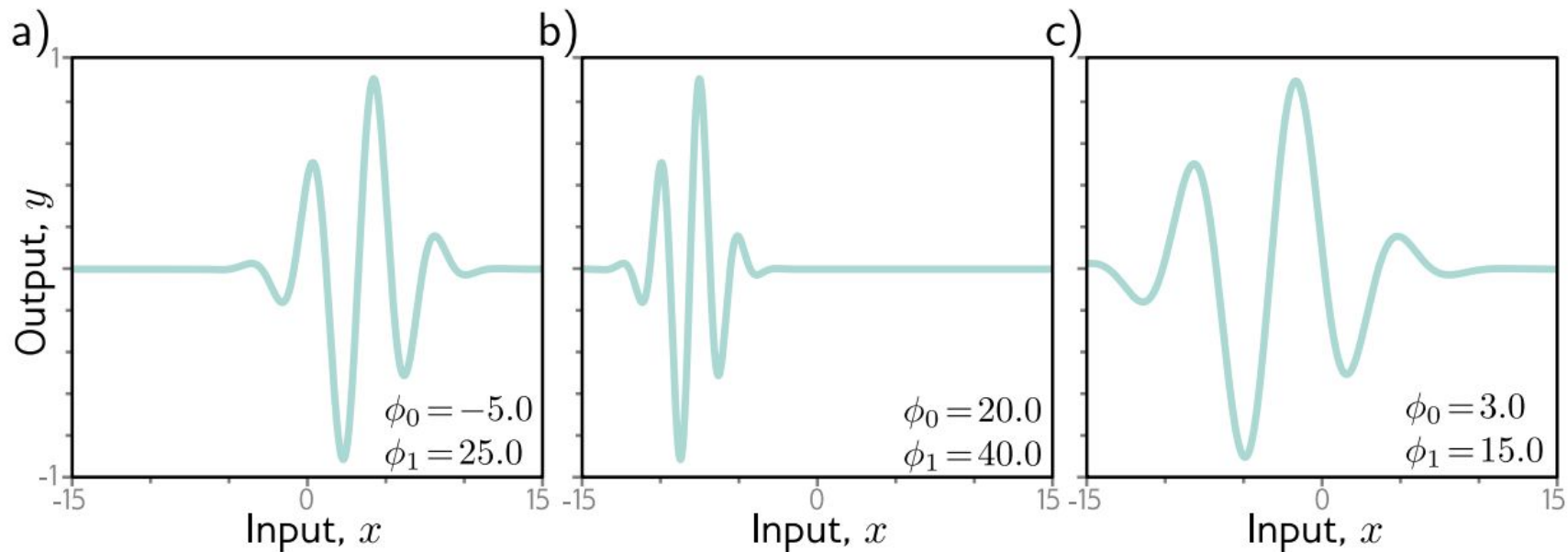


Figure 6.2 Gabor model. This nonlinear model maps scalar input x to scalar output y and has parameters $\phi = [\phi_0, \phi_1]^T$. It describes a sinusoidal function that decreases in amplitude with distance from its center. Parameter $\phi_0 \in \mathbb{R}$ determines the position of the center. As ϕ_0 increases, the function moves left. Parameter $\phi_1 \in \mathbb{R}^+$ squeezes the function along the x -axis relative to the center. As ϕ_1 increases, the function narrows. a–c) Model with different parameters. (Interactive figure)

Loss landscape of Gabor model

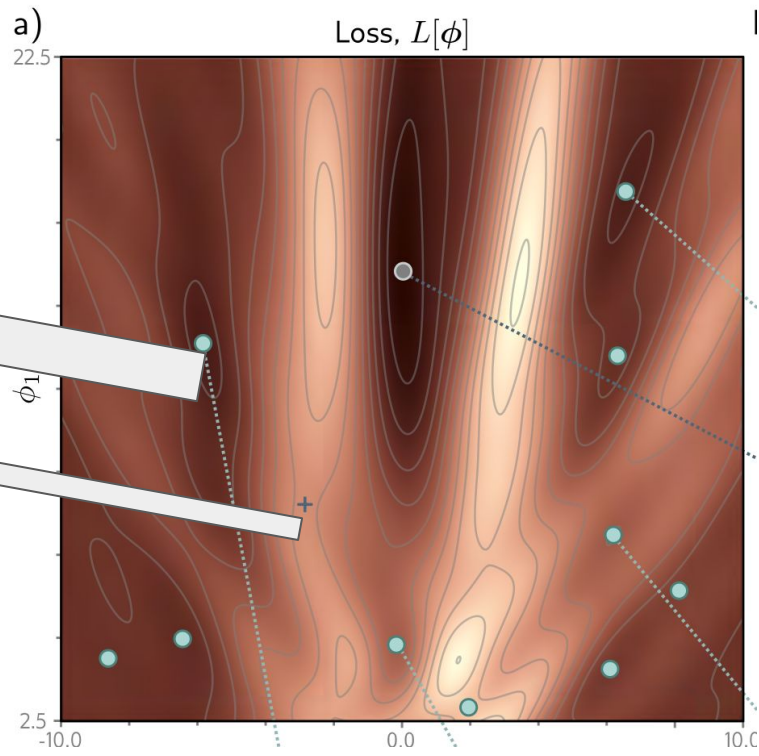
The fundamental difference between the linear loss landscape we saw earlier and this one is :

Lack of Convexity

This introduces:

1. Local Minimas
2. Saddle Points

What do you think will happen if we start step gradient descent at a saddle point ? What about at a local minima ?



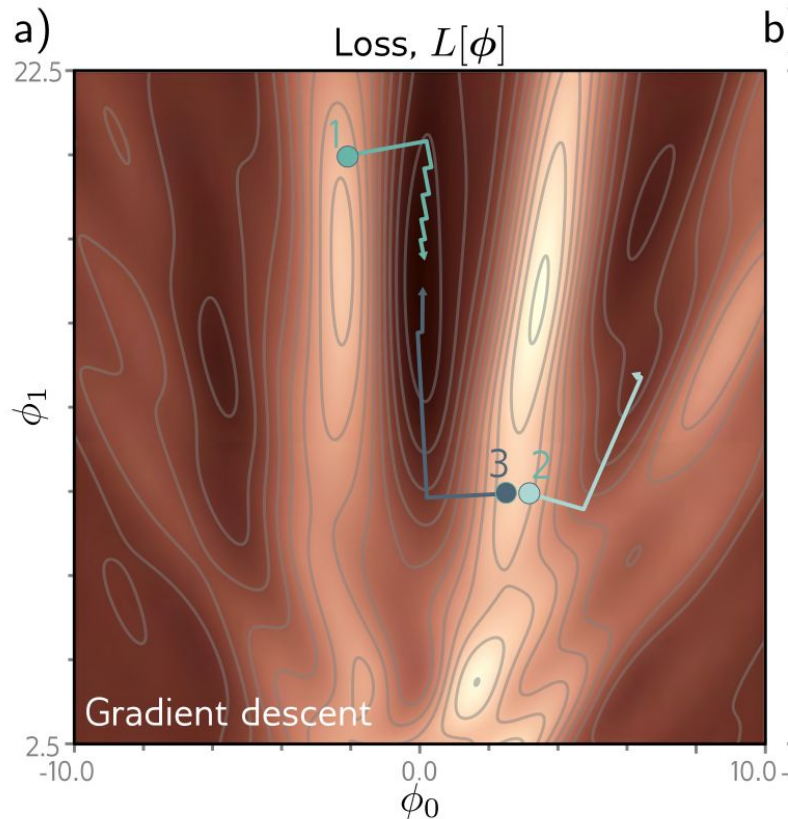
Performance of Step Gradient Descent

As long as the gradient descent algorithm is initialized in the right “valley” of the loss function (e.g., points 1 and 3),

the parameter estimate will move steadily toward the global minimum.

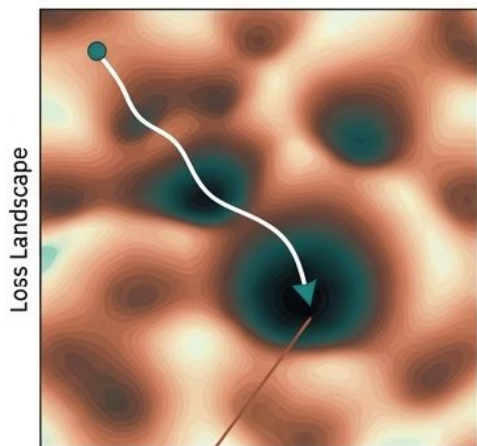
However, if it is initialized outside this valley (e.g., point 2), it will descend toward one of the local minima.

So how do we deal with this?



Stochastic Gradient Descent (SGD)

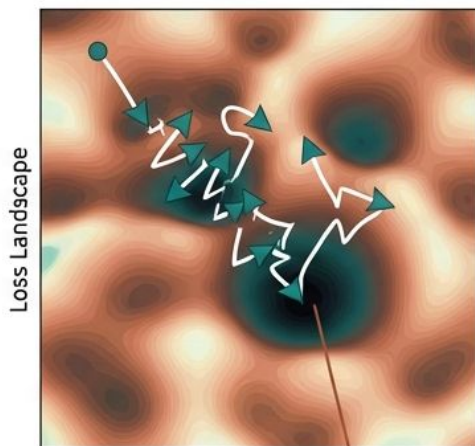
Standard GD



Loss Landscape

Local Minimum

SGD (Stochastic Gradient Descent)



Loss Landscape

Global Minimum

The Problem: Full-batch gradient descent is computationally expensive and deterministic—it gets stuck in the first valley it finds.

The Solution: Introduce randomness (Noise).

Mechanism:

1. Select a random **Minibatch** (B_t) of data.
2. Compute gradient only on this batch.
3. Update: $\phi_{t+1} \leftarrow \phi_t - \alpha \sum_{i \in B_t} \frac{\partial \ell_i}{\partial \phi}$

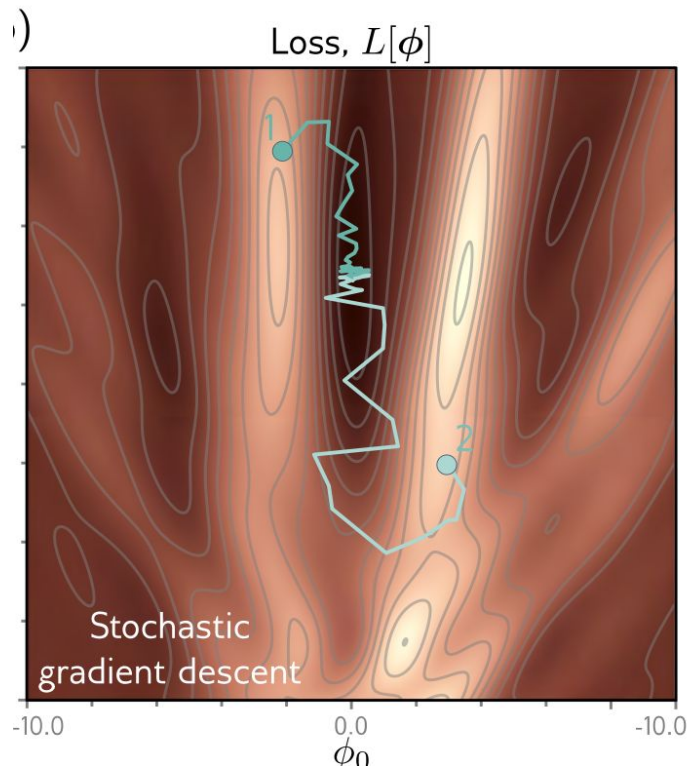
Outcome: The 'noisy' path can jump out of local minima.

Stochastic Gradient Descent

One of the main problems we faced is that the final destination of a gradient descent algorithm is entirely determined by the starting point.

Stochastic gradient descent (SGD) attempts to remedy this problem by adding some noise to the gradient at each step.

The solution still moves downhill on average, but at any given iteration, the direction chosen is not necessarily in the steepest downhill direction.

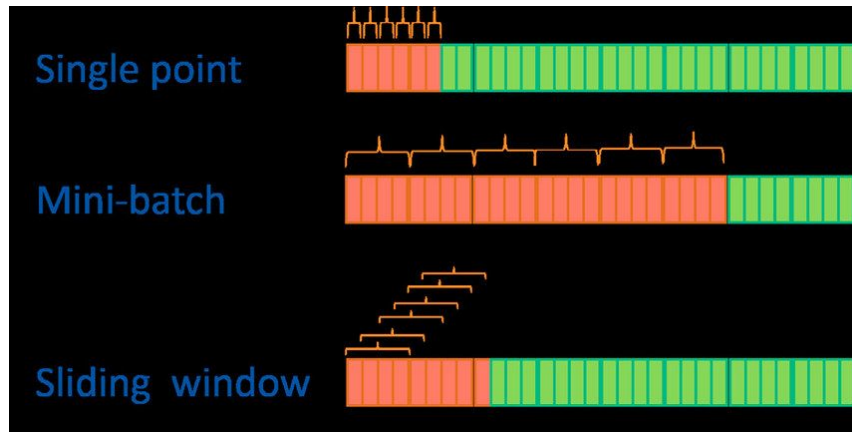


Introducing Randomness

At each iteration, the algorithm chooses a random subset of the training data and computes the gradient from these examples alone. This subset is known as a batch.

The batches are usually drawn from the dataset without replacement. The algorithm works through the training examples until it has used all the data, at which point it starts sampling from the full training dataset again.

A single pass through the entire training dataset is referred to as an epoch.



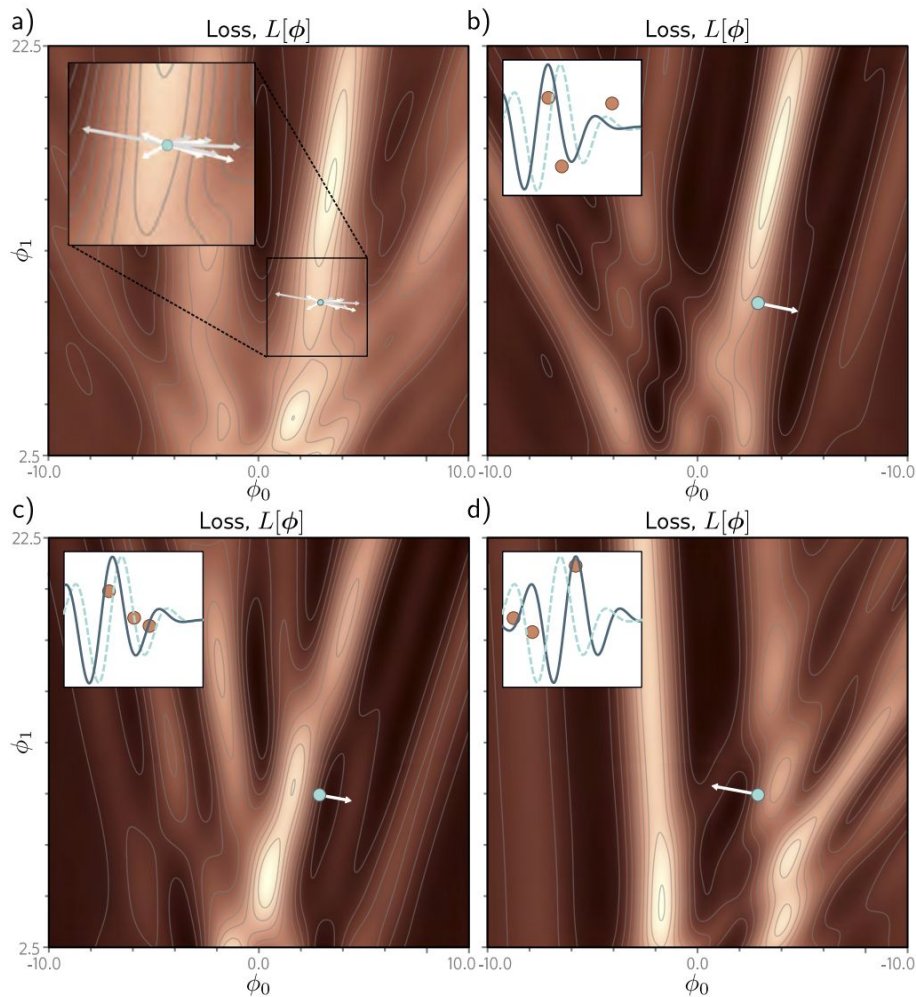


Figure 6.6 Alternative view of SGD for the Gabor model with a batch size of three.

a) Loss function for the entire training dataset. At each iteration, there is a probability distribution of possible parameter changes (inset shows samples). These correspond to different choices of the three batch elements.

b) Loss function for one possible batch. The SGD algorithm moves in the downhill direction on this function for a distance that is determined by the learning rate and the local gradient magnitude. The current model (dashed function in inset) changes to better fit the batch data (solid function).

c) A different batch creates a different loss function and results in a different update.

d) For this batch, the algorithm moves downhill with respect to the batch loss function but uphill with respect to the global loss function in panel (a). This is how SGD can escape local minima.

Understanding SGD (Mathematically)

$$\phi_{t+1} \leftarrow \phi_t - \alpha \cdot \sum_{i \in \mathcal{B}_t} \frac{\partial \ell_i[\phi_t]}{\partial \phi},$$

where $\mathcal{B}_t$ is a set containing
the indices of the input/output
pairs in the current batch

ℓ_i is the loss due to the i^{th}
pair

Batches, Epochs, and Dynamics

Minibatch: A subset of data used for one update step.

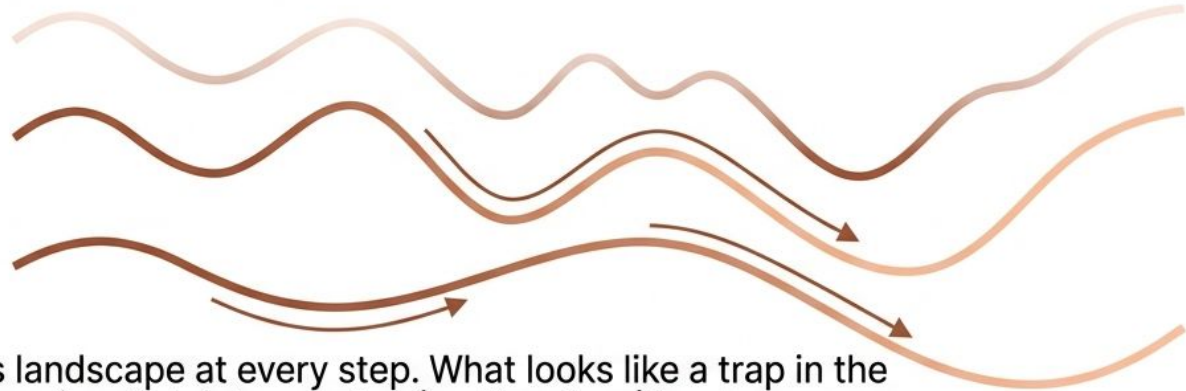
Epoch: A single pass through the entire training dataset.

Visualizing SGD Dynamics

Global Loss Surface
(Static) Jet Black

Batch 1 Loss Surface
IBM Plex Sans, Jet Black

Batch 2 Loss Surface
IBM Plex Sans, Jet Black



SGD views a different loss landscape at every step. What looks like a trap in the Global Loss might be a slope in a Batch Loss, **allowing the algorithm to escape.**
IBM Plex Sans, Jet Black

Problem 6.8* Can (non-stochastic) gradient descent with a *fixed* learning rate escape local minima?

A: No.

Reason: At a local minimum, $\nabla L = 0$.

Update rule: $\phi \leftarrow \phi - \alpha(0)$. The parameters stop changing.

Problem 6.9 We run the stochastic gradient descent algorithm for 1,000 iterations on a dataset of size 100 with a batch size of 20. For how many epochs did we train the model?

Step 1: Updates per Epoch = $100/20 = 5$ updates.

Step 2: Total Epochs = $1000/5 = 200$ Epochs.

Answer: 200 Epochs.

Pros of SGD

Because it draws training examples without replacement and iterates through the dataset, the training examples all still contribute equally.

It is less computationally expensive to compute the gradient from just a subset of the training data.

It can (in principle) escape local minima.

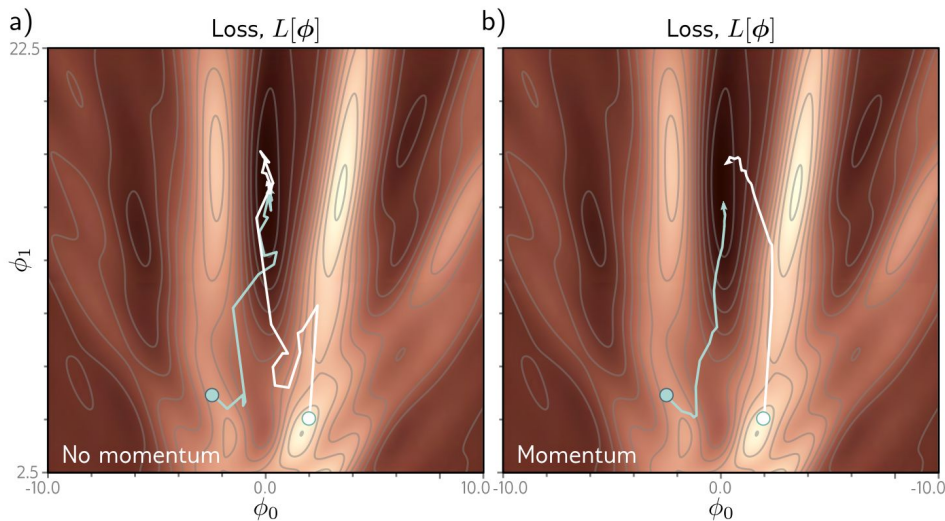
It reduces the chances of getting stuck near saddle points.

How do we make SGD better?

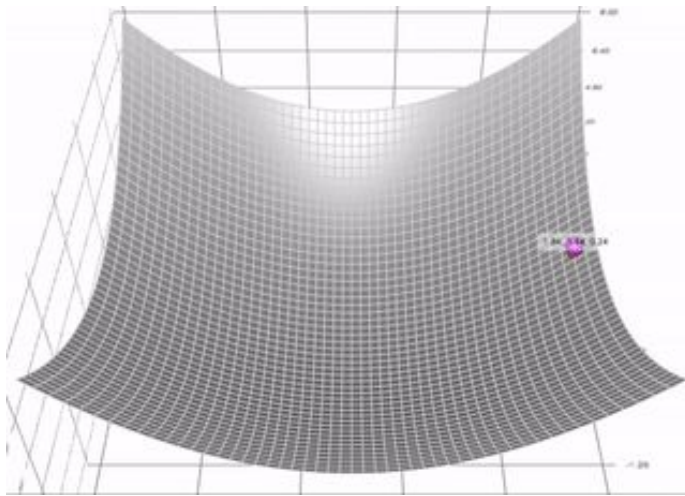
By using physics! (kind of)

A common modification to stochastic gradient descent is to add a momentum term.

Overall effect is a smoother trajectory and reduced oscillatory behavior in valleys.

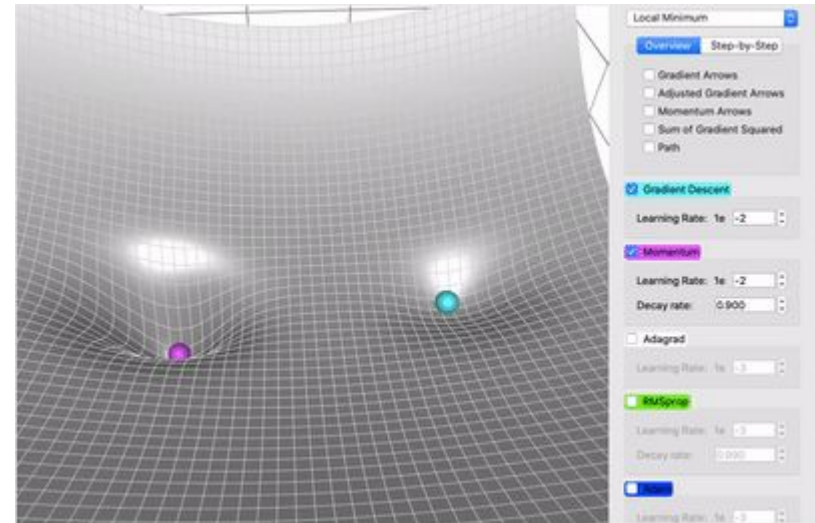


Adding momentum..



How momentum behaves

How momentum is useful



Momentum: Accelerating through Inertia

Concept: Accumulate a velocity vector v to smooth out oscillations.

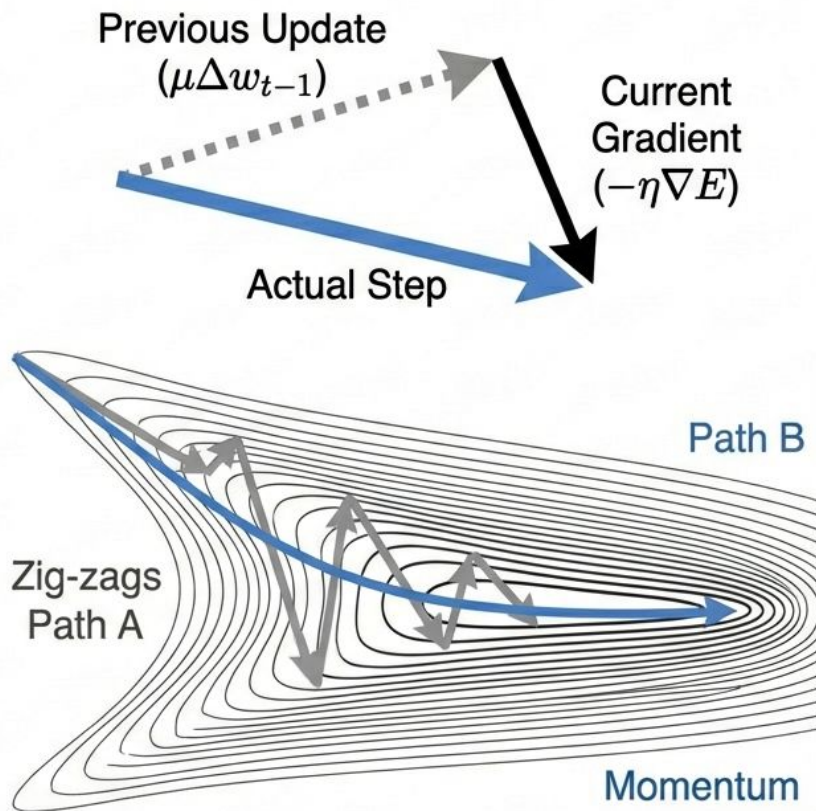
Update Rule:

$$\Delta w^{(\tau)} = \underbrace{-\eta \nabla E}_{\text{Gradient Step}} + \underbrace{\mu \Delta w^{(\tau-1)}}_{\text{Momentum Term}}.$$

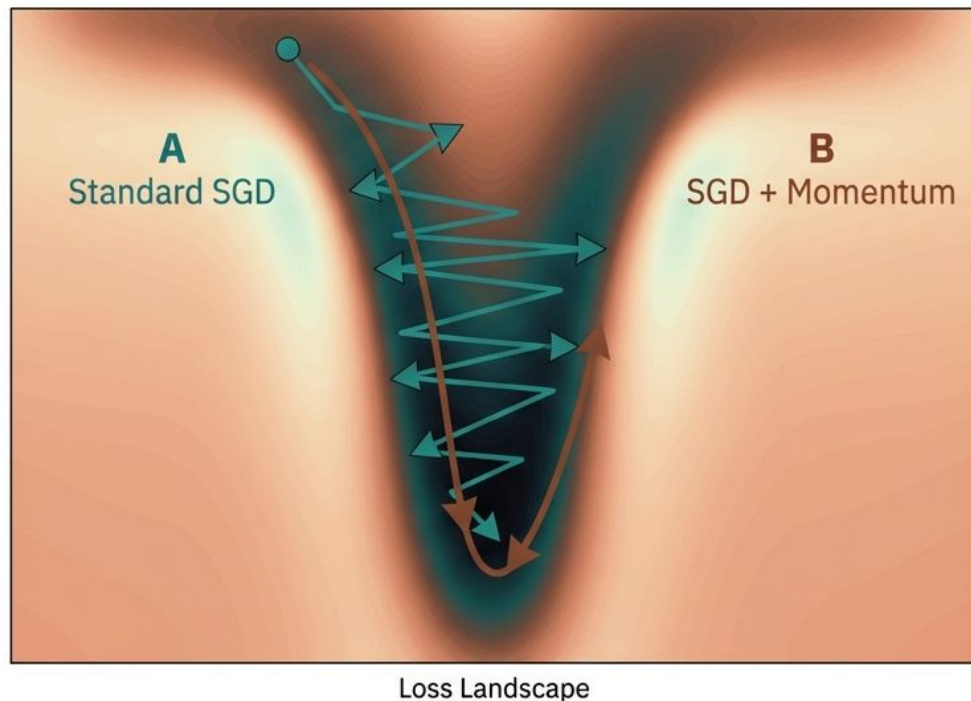
Effect:

- High curvature directions (oscillations) cancel out.
- Low curvature directions (flat) accumulate velocity.

Effective Rate: $\eta_{eff} \approx \frac{\eta}{1-\mu}.$



Momentum: Adding Inertia



The Issue: SGD oscillates in narrow valleys and moves slowly on flat surfaces.

The Fix: Accumulate velocity (m_t) from past gradients, like a heavy ball rolling down a hill.

The Equations

$$m_{t+1} \leftarrow \beta m_t + (1 - \beta) \text{Gradient}$$

$$\phi_{t+1} \leftarrow \phi_t - \alpha m_{t+1}$$

Parameter: $\beta \in [0, 1)$ controls the friction.

Understanding Momentum (Mathematically)

$$\mathbf{m}_{t+1} \leftarrow \beta \cdot \mathbf{m}_t + (1 - \beta) \sum_{i \in \mathcal{B}_t} \frac{\partial \ell_i[\phi_t]}{\partial \phi}$$

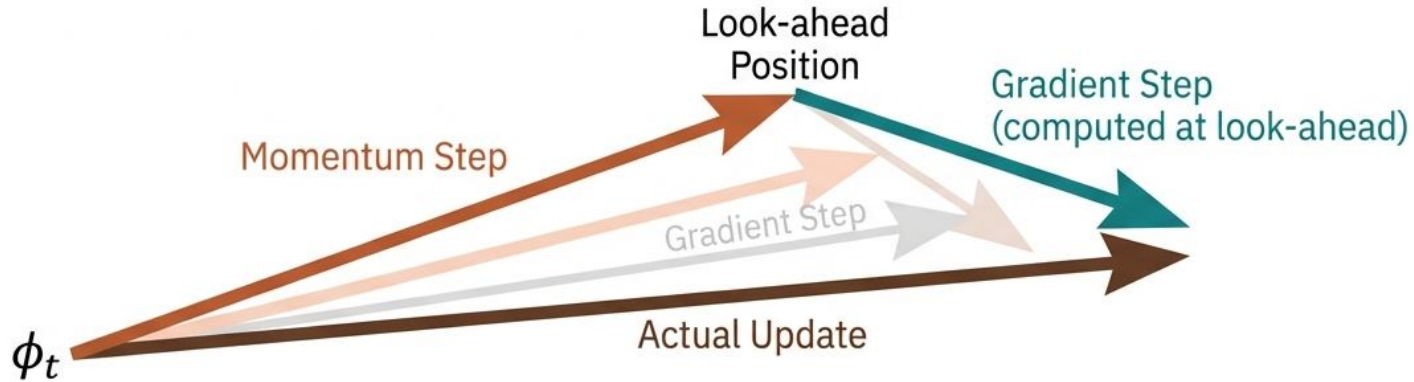
$$\phi_{t+1} \leftarrow \phi_t - \alpha \cdot \mathbf{m}_{t+1},$$

where $\mathbf{m}_t$ is the momentum

The recursive formulation of the momentum calculation means that the gradient step is an infinite weighted sum of all the previous gradients, where the weights get smaller as we move back in time.

The effective learning rate increases if all these gradients are aligned over multiple iterations but decreases if the gradient direction repeatedly changes as the terms in the sum cancel out.

Nesterov Accelerated Momentum



Concept: Standard momentum strikes blindly. Nesterov momentum “looks ahead” before calculating the gradient.

The Math: Gradients are evaluated at $(\phi_t - \alpha\beta m_t)$.

Benefit: Improved responsiveness and “braking” when approaching the valley floor.

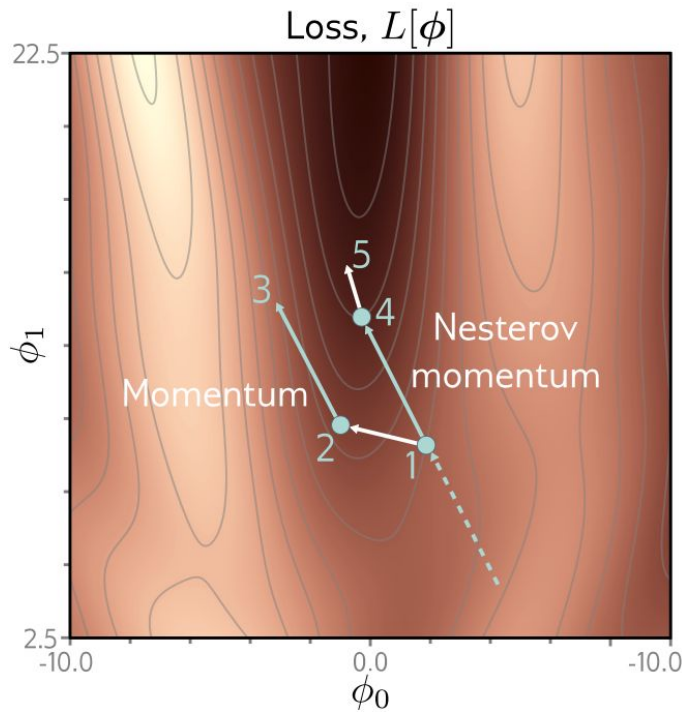
Nesterov accelerated momentum

The momentum term can be considered a coarse prediction of where the SGD algorithm will move next.

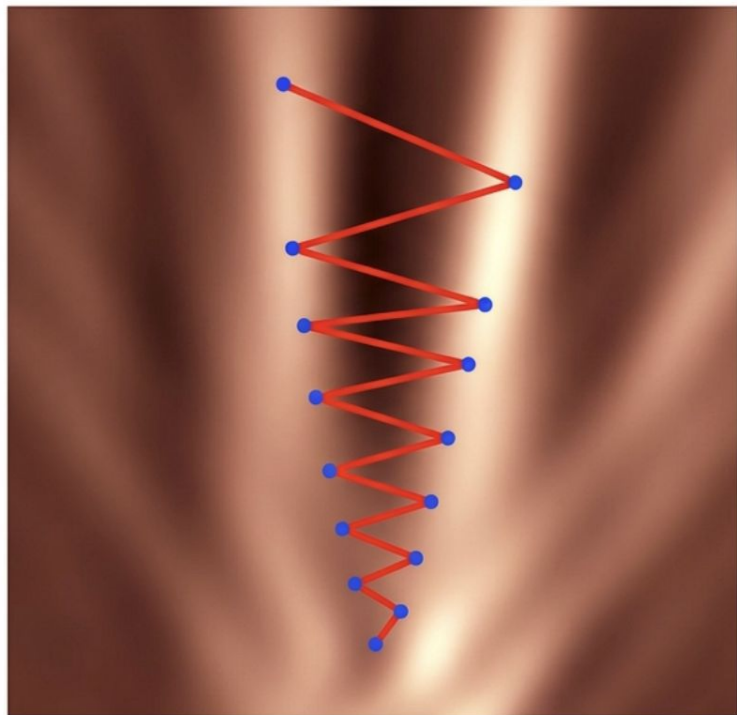
Nesterov accelerated momentum computes the gradients at this predicted point rather than at the current point.

$$\mathbf{m}_{t+1} \leftarrow \beta \cdot \mathbf{m}_t + (1 - \beta) \sum_{i \in \mathcal{B}_t} \frac{\partial \ell_i[\phi_t - \alpha \beta \cdot \mathbf{m}_t]}{\partial \phi}$$

$$\phi_{t+1} \leftarrow \phi_t - \alpha \cdot \mathbf{m}_{t+1},$$



Problem 6.7* The gradient descent trajectory for path 1 in figure 6.5a oscillates back and forth inefficiently as it moves down the valley toward the minimum. It's also notable that it turns at right angles to the previous direction at each step. Provide a qualitative explanation for these phenomena. Propose a solution that might help prevent this behavior.



Why? The gradient is always perpendicular to contour lines. In a narrow valley (steep walls, flat floor), the steepest descent points across the valley, not along it.

Consequence: Inefficient oscillation. We overshoot the valley floor repeatedly.

Solution: Add Momentum or normalize inputs.

Problem 6.10 Show that the momentum term $\mathbf{m}_t$ (equation 6.11) is an infinite weighted sum of the gradients at the previous iterations and derive an expression for the coefficients (weights) of that sum.

Momentum update: $m_{t+1} = \beta m_t + (1 - \beta)g_t$.

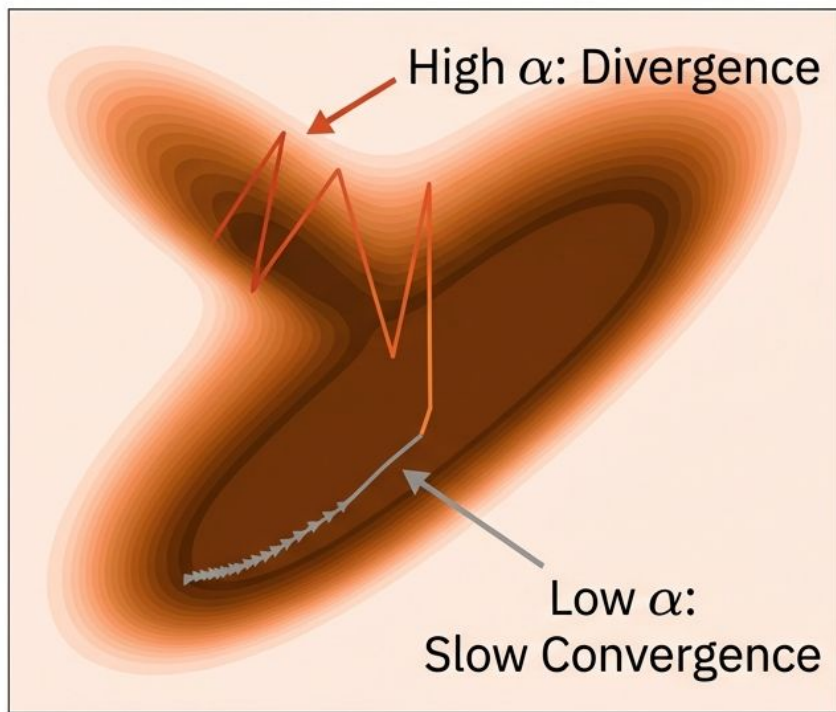
1. $m_1 = (1 - \beta)g_0$
2. $m_2 = \beta(1 - \beta)g_0 + (1 - \beta)g_1$
3. $m_3 = \beta^2(1 - \beta)g_0 + \beta(1 - \beta)g_1 + (1 - \beta)g_2$

$$m_{t+1} = (1 - \beta) \sum_{k=0}^t \beta^{t-k} g_k$$

Momentum is an infinite weighted sum of past gradients, with weights decaying exponentially over time.

Playing with the learning rate

The Scale Problem & Adaptive Learning



The Scenario: A Canyon Landscape (Steep walls, flat floor).

The Conflict: A single learning rate cannot handle both distinct scales.

The Solution: Adaptive Learning. We must normalize the gradients to move a consistent distance regardless of the slope steepness.

Learning Rate Schedules

Motivation: Start with high η for exploration, anneal to low η for fine convergence.

Linear Decay:

$$\eta(\tau) = (1 - \alpha)\eta_0 + \alpha\eta_{\text{final}}$$

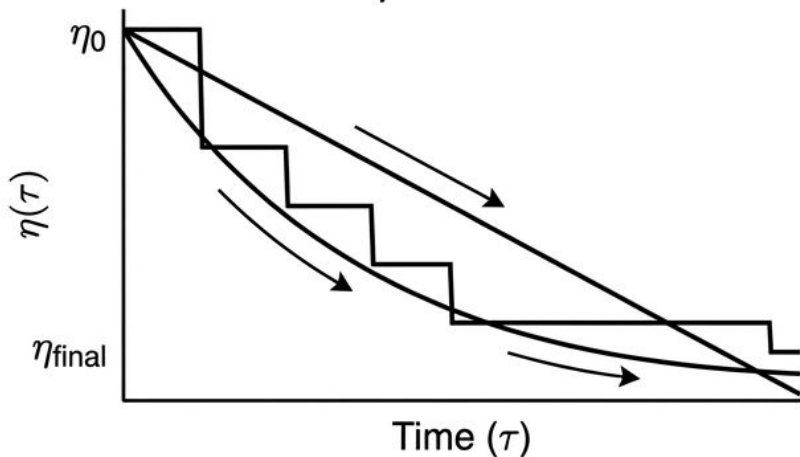
Power Law:

$$\eta(\tau) = \eta_0(1 + \tau/s)^{-c}$$

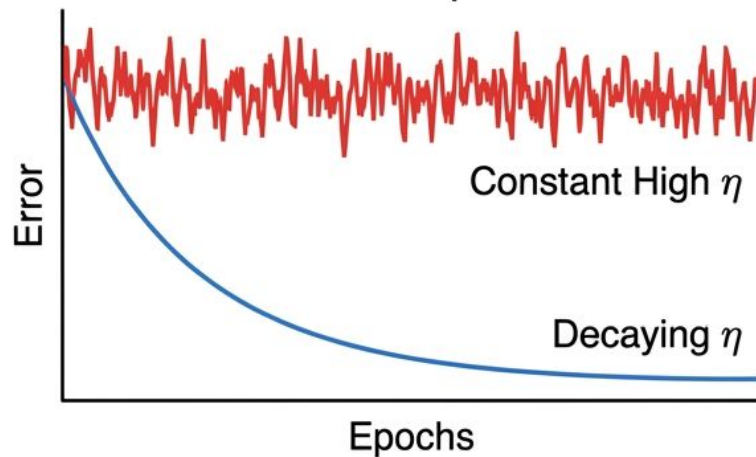
Exponential:

$$\eta(\tau) = \eta_0 c^{\tau/s}$$

η vs Time



Error vs Epochs





Epoch
000,000

Learning rate

0.03

Activation

ReLU

Regularization

L1

Regularization rate

0

Problem type

Classification

DATA

Which dataset do you want to use?



Ratio of training to test data: 50%

Noise: 15

Batch size: 11

REGENERATE

FEATURES

Which properties do you want to feed in?



+ - 3 HIDDEN LAYERS

+ -

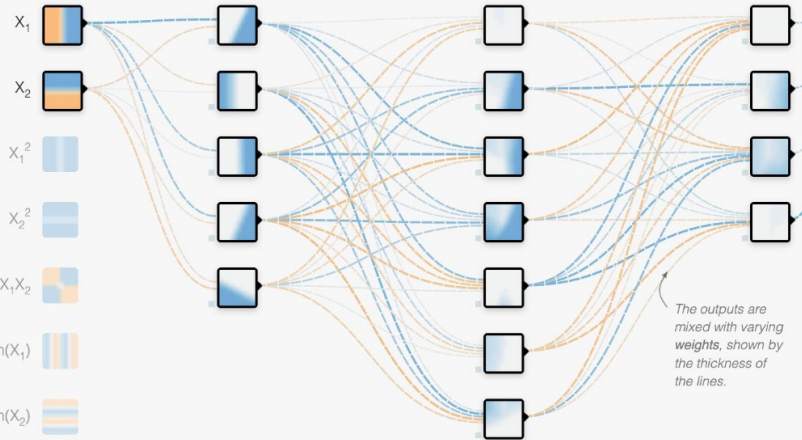
5 neurons

+ -

7 neurons

+ -

4 neurons

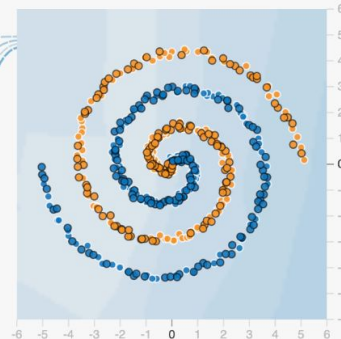


The outputs are mixed with varying weights, shown by the thickness of the lines.

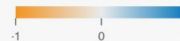
This is the output from one neuron. Hover to see it larger.

OUTPUT

Test loss 0.514
Training loss 0.526



Colors shows data, neuron and weight values.



☒ Show test data ☐ Discretize output

Adagrad(Adaptive Gradient)

$$G_t = G_{t-1} + \boxed{(\nabla_{\theta} J(\theta_t))^2}$$

→ Accumulate the square of gradient

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \cdot \nabla_{\theta} J(\theta_t)$$

Adaptive Learning Rates: RMSProp

Problem: A global learning rate η is inefficient when curvature varies per parameter.

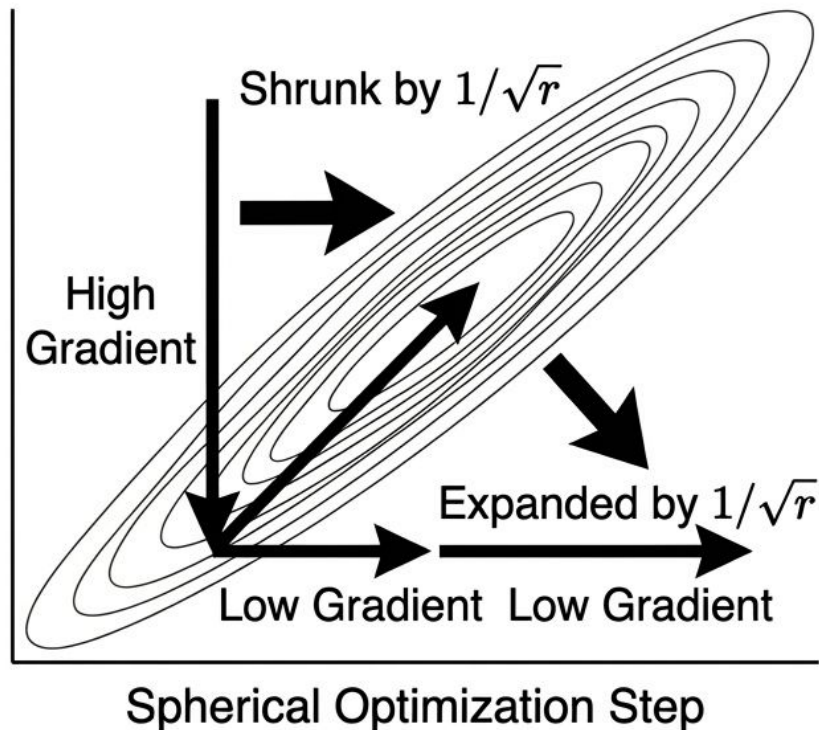
AdaGrad: Scales η by sum of squared past gradients.

Issue: Stops learning too early.

RMSProp: Uses exponentially weighted moving average.

$$r \leftarrow \beta r + (1 - \beta)(\nabla E)^2$$

$$w \leftarrow w - \frac{\eta}{\sqrt{r}} \nabla E$$



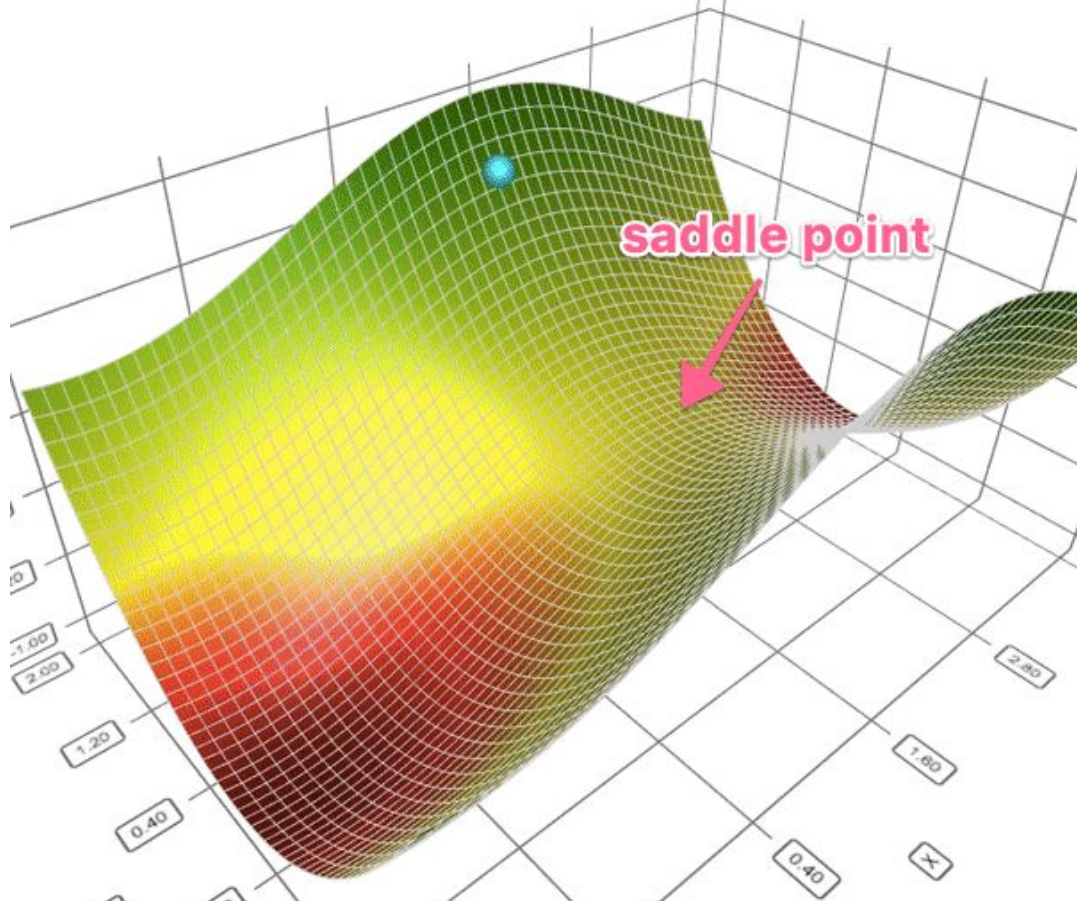
$$\text{SGD} \Rightarrow w_t = w_{t-1} - \eta \frac{\partial L}{\partial w_{t-1}}$$

$$\text{Adagrad} \Rightarrow w_t = w_{t-1} - \eta'_t \frac{\partial L}{\partial w_{t-1}}$$

$$\text{where } \eta'_t = \frac{\eta}{\sqrt{\alpha_t + \varepsilon}}$$

ε is a small +ve number to avoid divisibility by 0

$$\alpha_t = \sum_{i=1}^t \left(\frac{\partial L}{\partial w_{t-1}} \right)^2 \quad \text{summation of gradient square}$$



Overview Step-by-Step

- ☐ Gradient Arrows
- ☐ Adjusted Gradient Arrows
- ☐ Momentum Arrows
- ☐ Sum of Gradient Squared
- ☒ Path

☒ Gradient Descent

Learning Rate: 1e -2

☐ Momentum

Learning Rate: 1e -3

Decay rate: 0.800

☒ Adagrad

Learning Rate: 1e -1

☐ RMSprop

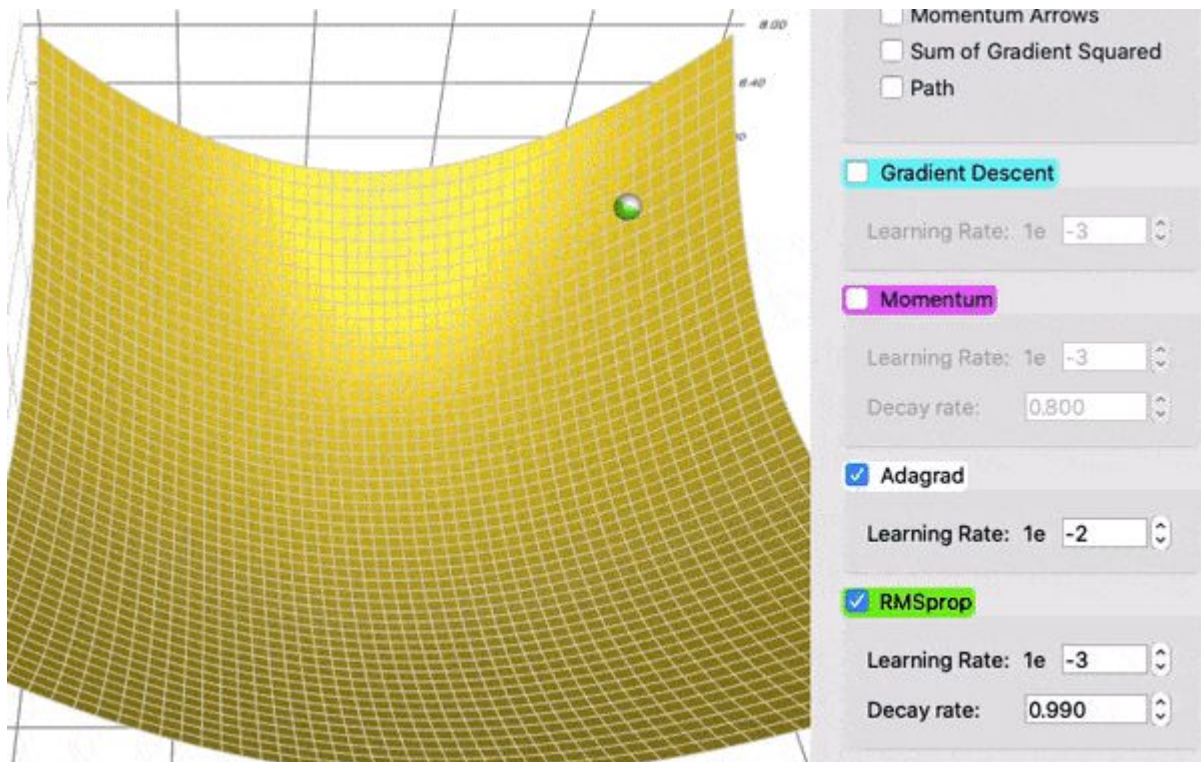
Learning Rate: 1e -3

Decay rate: 0.990

☐ Adam

Learning Rate: 1e -3

AdaGrad (white) vs. gradient descent (cyan) on a terrain with a saddle point. The learning rate of AdaGrad is set to be higher than that of gradient descent, but the point that AdaGrad's path is straighter stays largely true regardless of learning rate.



RMSProp (green) vs AdaGrad (white). The first run just shows the balls; the second run also shows the sum of gradient squared represented by the squares.

The Pragmatist: RMSProp

For each Parameter w^j

(j subscript dropped for clarity)

$$\nu_t = \rho \nu_{t-1} + (1 - \rho) * g_t^2$$

$$\Delta \omega_t = - \frac{\eta}{\sqrt{\nu_t} + \epsilon} * g_t$$

$$\omega_{t+1} = \omega_t + \Delta \omega_t$$

η : Initial Learning rate

ν_t : Exponential Average of squares of gradients

g_t : Gradient at time t along ω^j

Introduction of Forgetfulness

Uses an exponentially weighted moving average instead of a simple sum.

- ▶ Discards distant history.
- ▶ Adapts locally to current terrain.
- ▶ Prevents premature convergence.

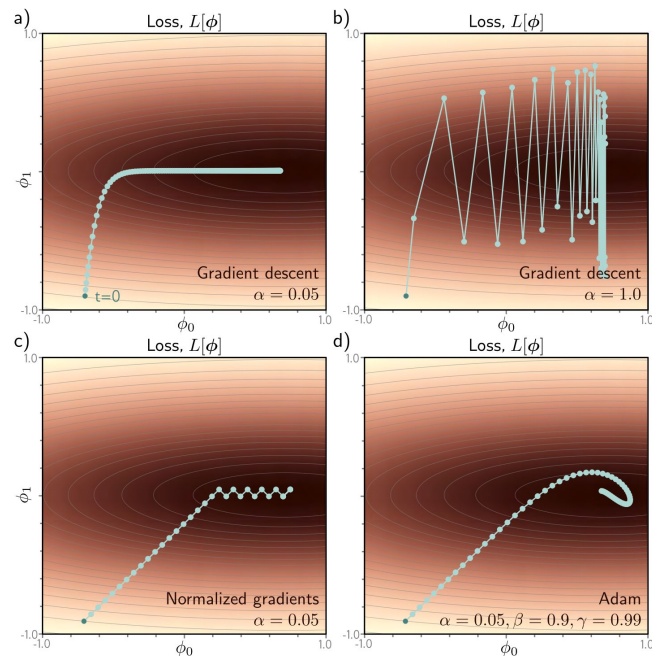
**Combining all the goodness:
ADAM**

Adaptive moment estimation (Adam)

Gradient descent with a fixed step size has the following undesirable property:

1. *It makes large adjustments to parameters associated with large gradients (where perhaps we should be more cautious) and*
2. *Small adjustments to parameters associated with small gradients (where perhaps we should explore further).*

A straightforward approach is to normalize the gradients so that we move a fixed distance (governed by the learning rate) in each direction.



Adam, takes this idea and adds momentum to both the estimate of the gradient and the squared gradient

Adam: Adaptive Moment Estimation

Momentum (First Moment) + RMSProp (Second Moment)

1. Estimate Mean (m):

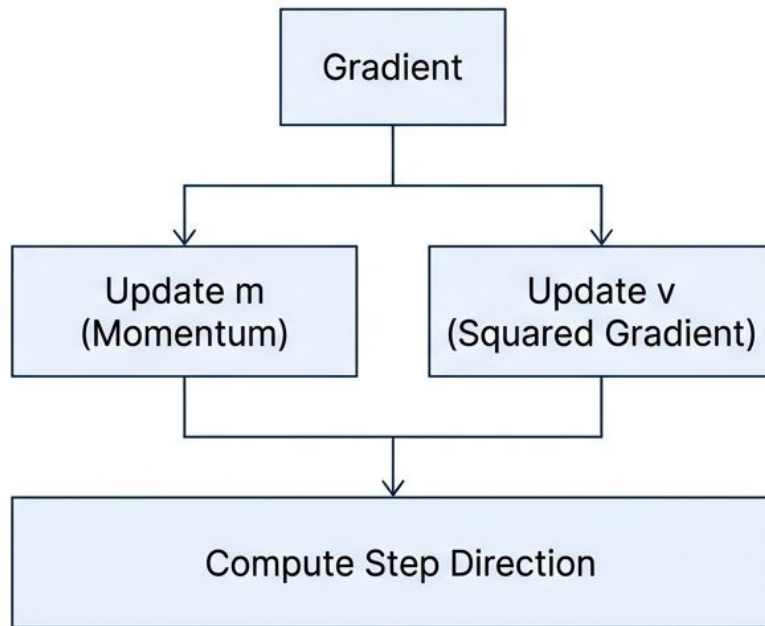
$$m_{t+1} \leftarrow \beta \cdot m_t + (1 - \beta) \frac{\partial L}{\partial \phi}$$

2. Estimate Variance (v):

$$v_{t+1} \leftarrow \gamma \cdot v_t + (1 - \gamma) \left(\frac{\partial L}{\partial \phi} \right)^2$$

3. Update:

$$\phi_{t+1} \leftarrow \phi_t - \alpha \cdot \frac{m_{t+1}}{\sqrt{v_{t+1}} + \epsilon}$$



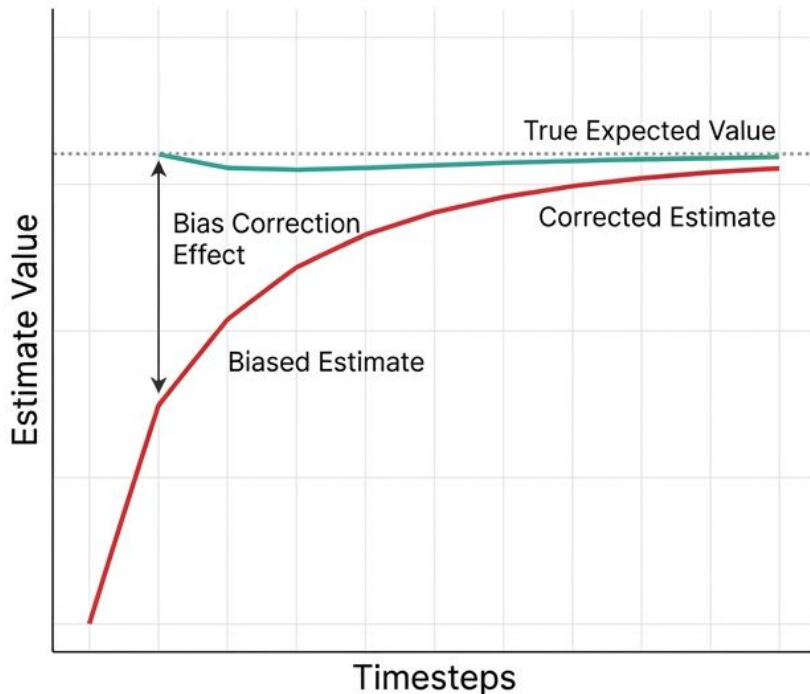
Adam: Bias Correction

Problem: m and v are initialized to 0. Early estimates are biased toward 0 (vanishing steps).

Solution: Rescale based on timestep t .

$$\hat{m}_{t+1} \leftarrow \frac{m_{t+1}}{1 - \beta^{t+1}}$$

$$\hat{v}_{t+1} \leftarrow \frac{v_{t+1}}{1 - \gamma^{t+1}}$$



Hyperparameters

Configuration settings chosen before training.



Learning Rate (α)

The most critical setting. Too high = diverge. Too low = slow.



Batch Size

Trade-off: Speed (Large Batch) vs. Exploration Noise (Small Batch).



Momentum Coefficients

Standard values: $\beta = 0.9$, $\gamma = 0.99$.



Learning Rate Schedules

Decaying α over time. Explore early (high α), fine-tune late (low α).

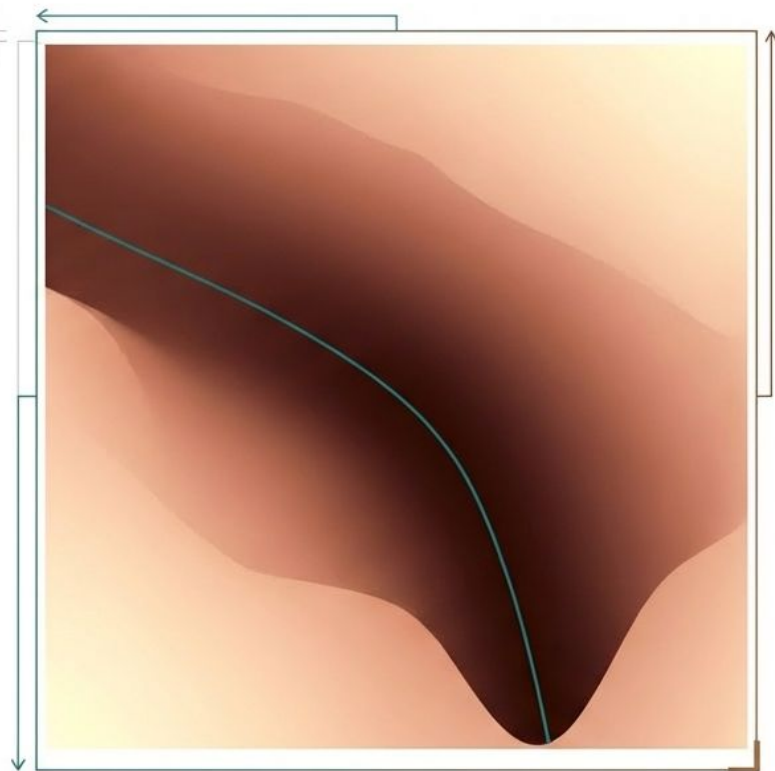
Optimizer Comparison

SGD: Noisy, slow convergence, easily distracted.

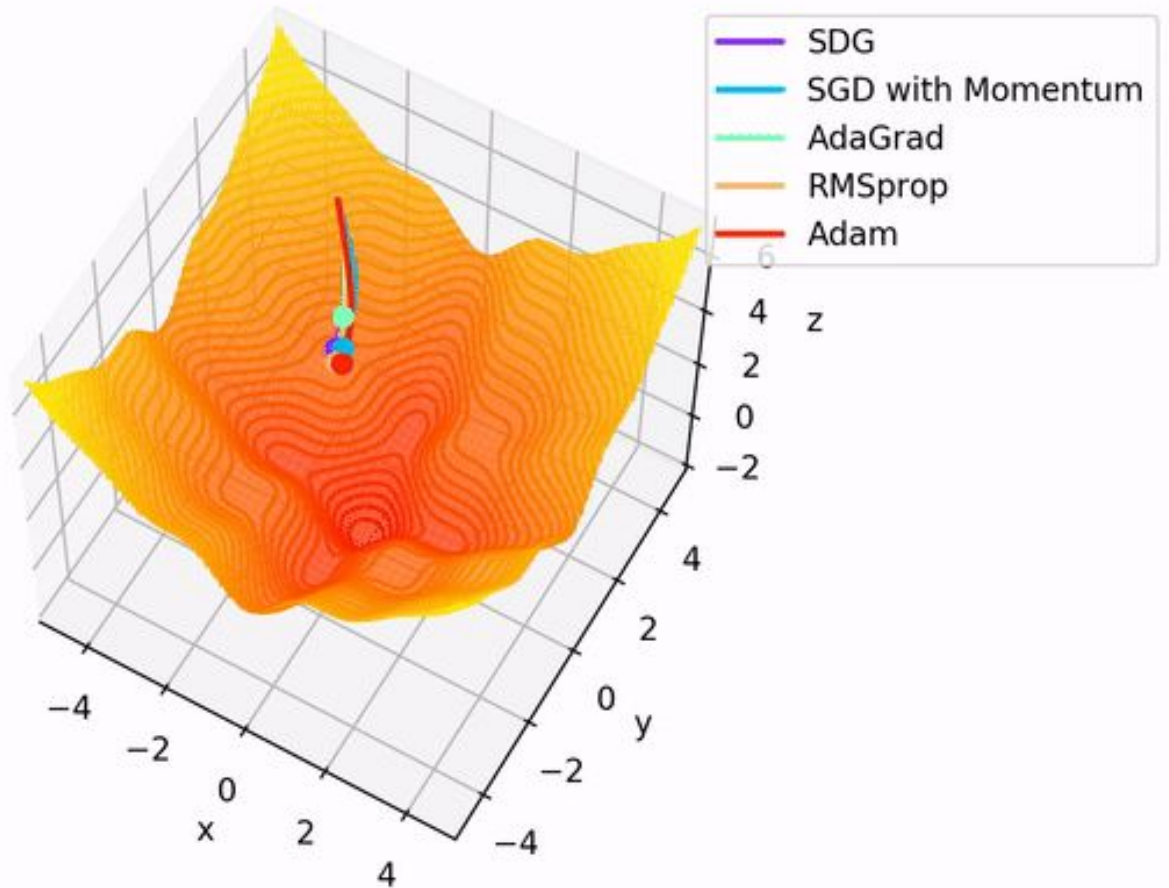
Momentum: Faster, but can overshoot.

Adam: Fast, stable, and handles the "Canyon" problem elegantly by normalizing the path.

Verdict: Adam generally works well “out of the box” across a broad range of architectures.

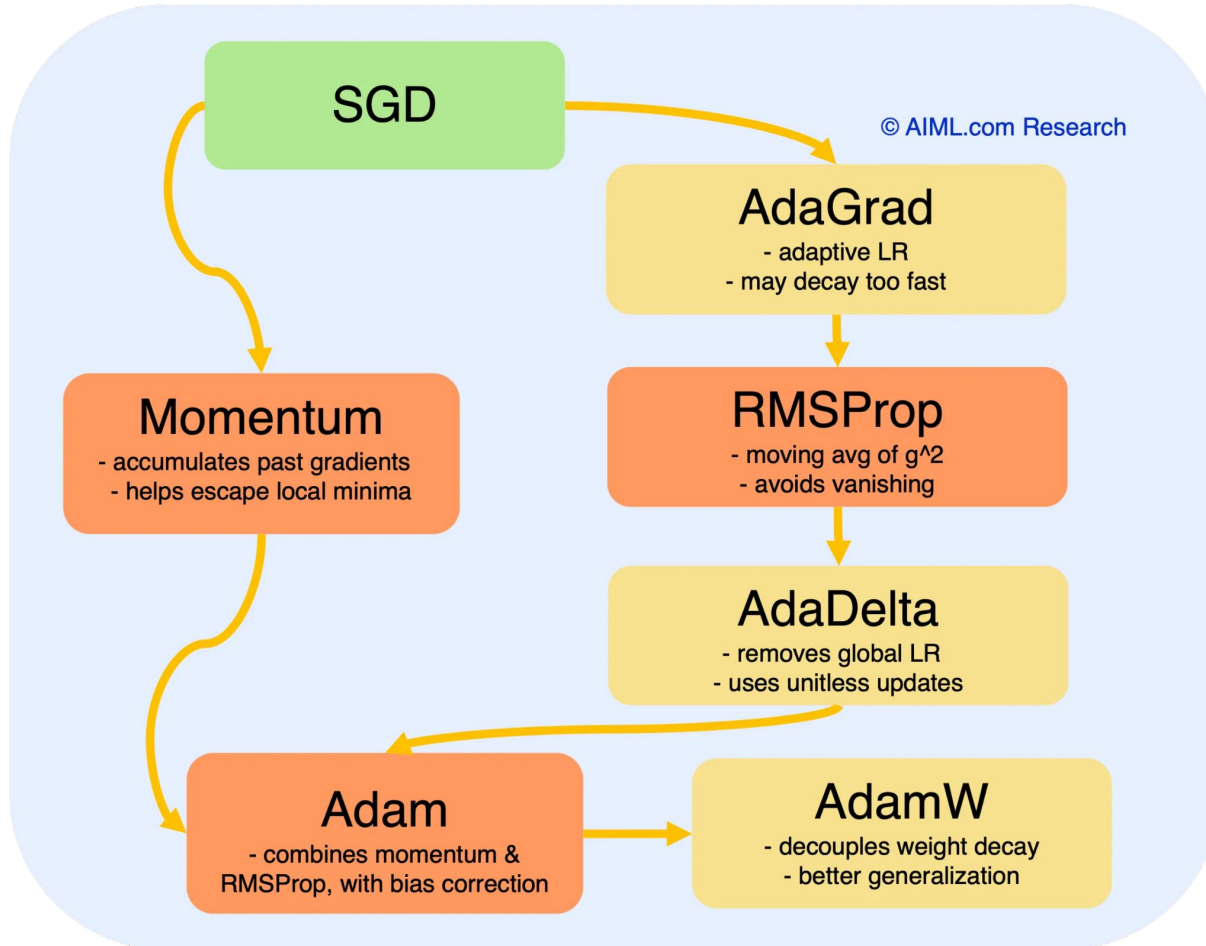


Optimizer Comparison



[GitHub - lilipads/gradient_descent_viz](#): interactive visualization of 5 popular gradient descent methods with step-by-step illustration and hyperparameter tuning UI

Evolution of Optimizers



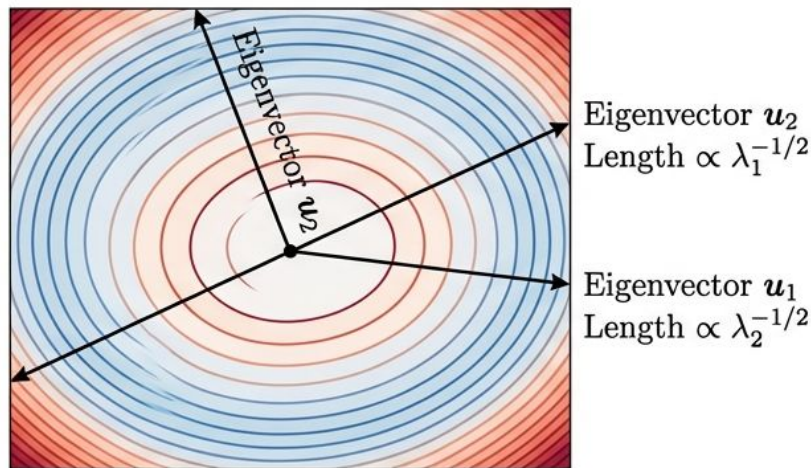
What about Second Order Methods..?

Geometry of the Minimum: Eigenvalue Analysis

Near a minimum w^* , the gradient vanishes. The error surface is determined by the Hessian H .

Eigenvalue Decomposition: $Hu_i = \lambda_i u_i$.

Error relative to minimum: $E(w) = E(w^*) + \frac{1}{2} \sum_i \lambda_i \alpha_i^2$.



Critical Conditions

1. **Positive Definite** ($\forall \lambda_i > 0$):
Local Minimum (The Bowl).
2. **Negative Definite** ($\forall \lambda_i < 0$):
Local Maximum (The Hill).
3. **Indefinite (Mixed signs)**:
Saddle Point.

Hessian

The **Jacobian** of the derivatives $\partial f / \partial x_1, \partial f / \partial x_2, \dots, \partial f / \partial x_n$ of a function $f(x_1, x_2, \dots, x_n)$ with respect to $x_1, x_2, \dots, x_n$ is called the Hessian (or Hessian matrix) H of f , i.e.,

$$H f(x_1, x_2, \dots, x_n) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \frac{\partial^2 f}{\partial x_1 \partial x_3} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \frac{\partial^2 f}{\partial x_2 \partial x_3} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \frac{\partial^2 f}{\partial x_n \partial x_3} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}.$$

As in the case of the **Jacobian**, the term "Hessian" unfortunately appears to be used both to refer to this matrix and to the determinant of this matrix (Gradshteyn and Ryzhik 2000, p. 1069).

In the **second derivative test** for determining extrema of a function $f(x, y)$, the

Theoretical Limit: The Hessian Wall

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

$O(n^3)$

Computational Complexity

Why Second-Order Methods Fail

Newton's Method requires inverting the Hessian matrix.



Hessian Size: Memory and compute requirements are impossible.



Saddle Trap: Second-order methods are often attracted to saddle points.



Winning Recipe: Modern deep learning relies on First-Order adaptive methods (Adam) + smart initialization (Xavier/He).

Problem 6.11: The Curse of Dimensionality

Question: What are the dimensions of the Hessian for a model with 1,000,000 parameters?

$$N = 10^6 \quad (1 \text{ Million Parameters})$$

$$\text{Hessian Size} = N \times N$$

$$10^6 \times 10^6 = 10^{12} \text{ elements}$$

Storage (Float32): **4 Terabytes.**

Operation: Matrix Inversion is $O(N^3)$.

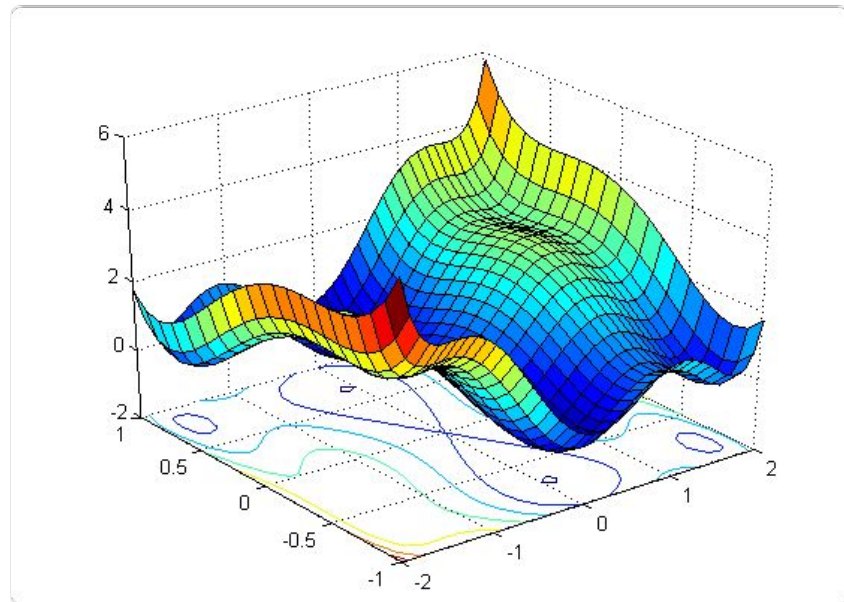
Result: Second-Order Optimization is computationally intractable for Deep Learning.

The Myth of Local Minima

Old Fear vs. New Reality

Researchers once feared high-cost local minima. We now know they are rare in high dimensions.

- ▶ Most minima have very low cost.
- ▶ Identifying the "true" global minimum is often unnecessary.



Recap

1. Convexity
2. Step Gradient Descent
3. Gabor Model
4. Step Gradient Descent performance
on non-convex loss landscape
5. Stochastic Gradient Descent
6. Momentum
7. Moment Estimation (Adam)

Parameter Initialization

Symmetry Breaking & He Initialization

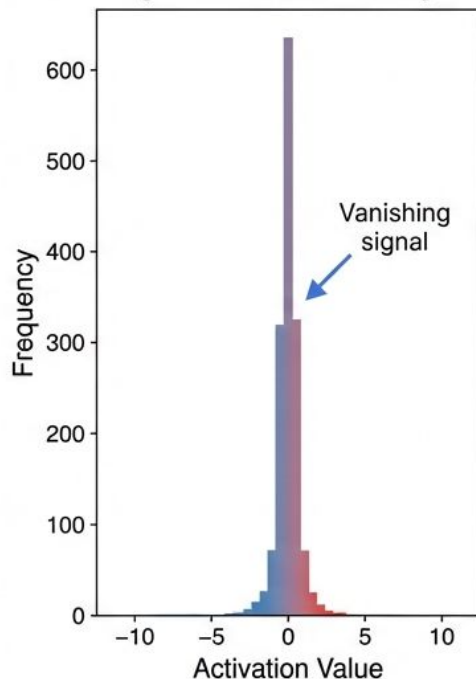
The Problem: Initializing weights to zero causes identical updates (symmetry). Random initialization is required.

He Initialization Strategy:

Goal: Maintain variance of activations across layers. For a layer with M inputs and ReLU activation:

$$\text{Var}(w) = \frac{2}{M} \Rightarrow \epsilon = \sqrt{\frac{2}{M}}$$

Poor Initialization
(Small Random)



He Initialization

